

Технологии программирования

**3 Закона
Робототехники**



Вопрос

Парадигмы программирования

- Императивное (asm)
- Функциональное (C, paskal, Ada old)
- Логическое (SML)
- Декларативное (Html, Qml)
- Объектно ориентированное (C++, Java, Python, Ada)

Базовый принцип

Объектно-ориентированное программирование

Парадигма программирования, в которой основной концепцией является понятие объекта, отождествленная с предметной областью.

Система рассматривается как набор объектов предметной области, взаимодействующие между собой. Каждый объект обладает тремя составляющими:

- Состояние — это набор всех его полей и их значений.
- Поведение — это набор всех методов класса объекта.
- Идентичность — это то, что отличает один объект класса от другого объекта класса.

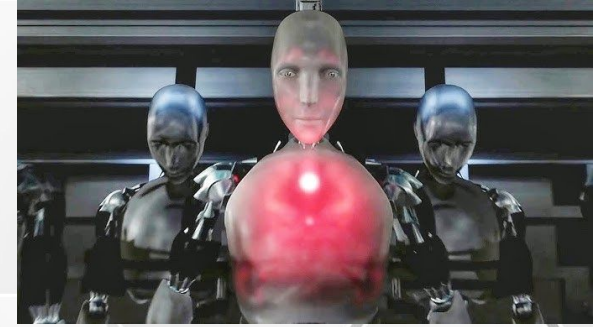


Вопрос

Принципы ООП

- Инкапсуляция
- Наследование
- Полиморфизм
- *Абстракция/абстрагирование*
- *Модульность*

Закон 1



Инкапсуляция

Принцип, согласно которому любой класс и в более широком смысле – любая часть системы должны рассматриваться как «черный ящик». Пользователь класса или подсистемы должен взаимодействовать только с интерфейсом и не вникать во внутреннюю реализацию.

Последствия

Если класс А обращается к полям класса В напрямую, это приводит к тому, что класс А завязывается на внутреннее устройство класса В. Попытка изменить внутреннее устройство класса В может привести к изменению класса А.

К тому же класс А не просто так работает с полями класса В, он формирует некую бизнес-логику совместной работы с классом В. То есть часть логики работы с состоянием класса В лежит в классе А, и класс В становится неполноценным, а его использование без класса А может быть невозможным. Если принцип нарушается для всей системы, то она становится неделимой.

Недооцененный принцип, многие считают что следование ему означает исключительно “приватизацию” полей класса.

Закон 2

Наследование

Возможность порождать один класс от другого с сохранением всех свойств и методов класса-предка, добавляя при необходимости новые свойства и методы

Иерархия классов должна проектироваться чтобы выразить свойство реального мира как иерархичность. Не стоит воспринимать наследование как способ переиспользовать код, наследуя объекты исходя из косвенных признаков, например унаследовав машину от холодильника, потому что у обоих предметов есть ручка.

Наследование является очень сильной связью. Для уменьшения количества уровней наследования рекомендуется строить дерево «снизу-вверх».



Переоцененный принцип. У идеального программиста дерево наследования уходит в бесконечность и заканчивается абсолютно пустым объектом

Закон 3

Полиморфизм

Возможность использовать классы потомки в контексте, который был предназначен для класса – предка.

Взаимодействие с объектами классов ведется только на уровне их интерфейсов, игнорируя внутреннее устройство. Нам достаточно интерфейса и нам всё равно что за объект за ним скрывается, важно только то, что мы можем вызвать любую функцию интерфейса и она будет выполнена в соответствии с нашими ожиданиями (теоретически...)



Закон 4

Абстракция

При проектировании класса следует сосредоточить внимание на необходимой для данной задачи информации, и исключить малозначимую или лишнюю.

Не следует усложнять структуру класса в угоду копирования реального объекта или во имя демонстрации и воплощения полноты идеи. Необходимо выделить только важное, необходимое.

Бритва Оккама

Принцип схожий с принципом экономии мышления: “Любое определение должно быть предельно простым” - “Не стоит плодить сущностей сверх необходимого”

KISS

Keep it Simple, Stupid - принцип построения программ утверждающий что большинство систем работают лучше всего, если они остаются простыми, а не усложняются.

BMC США 1960 г.

Принцип до сих пор оспаривается. Программисты “старой школы” считают его включение в принципы ООП - ошибкой.



Принципы позитронного мышления

Interface

Описание методов и полей для внешнего взаимодействия, достаточных для работы с классами, имеющими этот интерфейс.

```
class SomeInterface;
```

Class/struct

Описание методов и полей класса которые реализуют логику работы класса (бизнес-логику).

```
class SomeClass;
```

Object

Совокупность данных класса характеризующих определенное состояние объекта класса.

```
SomeClass myObject;
```



Конструкция

```
class SomeClass
{
public:
    SomeClass();           //Конструктор
    ~SomeClass();          //Деструктор
    int doSomething();      //Метод
private:
    double m_myInternalState; //Поле
}
```



Конструкторы

- Специальные функции, которые вызываются при создании объекта класса, могут быть вызваны явно, или через лист инициализации.
- В классе может быть больше одного.
- Имя метода то же что и имя класса.

Деструктор

- Специальная функция, которые вызываются при удалении объекта класса, Явно вызывать можно, но не нужно, класс перейдет в невалидное состояние.
- Только один на класс.
- Имя метода то же что и имя класса с тильдой '~'.

Методы (включая операторы)

- Функции для работы с классом, имеют доступ ко всем полям класса, вне зависимости от модификаторов доступа.

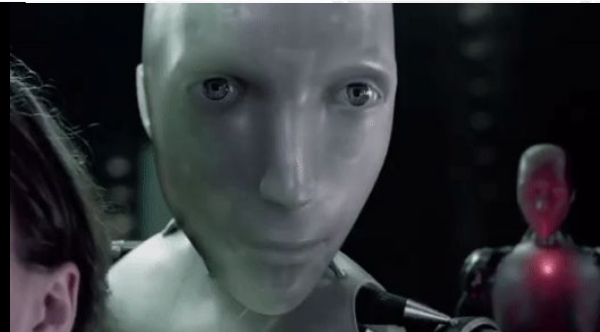
Поля

- Параметры определяющие состояние объекта класса.

Ставить m_ (member) перед именем поля класса не является обязательным, но полезно для чтения

Ограничения

```
class SomeClass
{
public:
    friend class MyBestFriend;
    void interfaceMethod();
protected:
    int forChildren();
private:
    double myInternalState;
}
```



Public

Интерфейсная часть - наборы методов и полей, доступных для работы с классом.

Интерфейс должен быть достаточным, чтобы не обладая информацией о внутреннем устройстве класса мы могли взаимодействовать с ним, для выполнения поставленных задач.

Protected

Защищенный блок, доступный только наследникам, набор функций и полей доступный по цепочке наследования, интерфейс “только для своих”.

Private

Внутренняя структура класса, здесь должны быть скрыты все алгоритмы взаимодействий и связи между данными класса, определяющими его состояние.

Friend

Сторонний класс или функция имеющий доступ ко всем полям класса.

**По умолчанию поля класса приватные. структуры - публичные
friend - прямое нарушение принципа инкапсуляции и признак плохого дизайна**

Вопрос

```
class SomeClass
{
}
```

Поля по умолчанию

Поля по умолчанию присутствуют в классе, компилятор сам добавит их, если не специфицировано обратное.

Конструкторы

- Дефолтный - существует пока в классе не объявлен любой другой конструктор
- Копирования
- Перемещения

Операторы

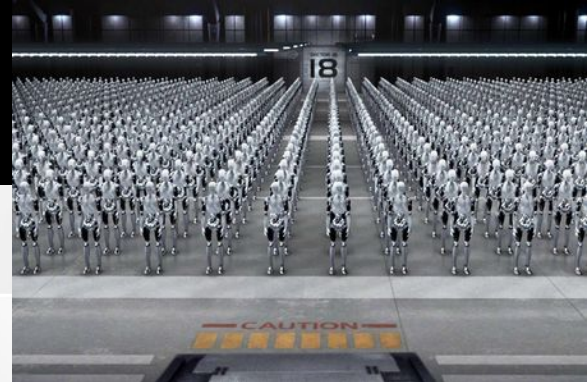
- Присваивания

Деструкторы

- Пустой деструктор

Фабрика

```
class SomeClass
{
public:
    int m_someField1 = 0;
    int m_someField2 = 10;
```



Конструктор по умолчанию

- Инициализирует поля класса

Обычный конструктор

```
SomeClass(int parameters) : m_someField1(parameters), m_someField2(45) { ... };
имя ctor'a | параметры | список инициализации полей класса | тело |
```

- При наличии обычного конструктора конструктор по умолчанию не создается
- Отсутствующие в списке поля будут инициализированы в соответствии с декларацией

Делегирующий конструктор

```
SomeClass(int param, float fparam) : SomeClass(param) { ... };
имя ctor'a | параметры | вызов ctor'a | тело |
```

- Поля в делегирующем конструкторе инициализировать нельзя

Конструкторы копирования и перемещения

```
SomeClass(const SomeClass& original) {} // копирование
SomeClass(SomeClass&& victim) {} // перемещение
```


Ликвидация

Деструктор по умолчанию

- Пустая функция, вызывает деструкторы всех полей класса.

Обычный деструктор

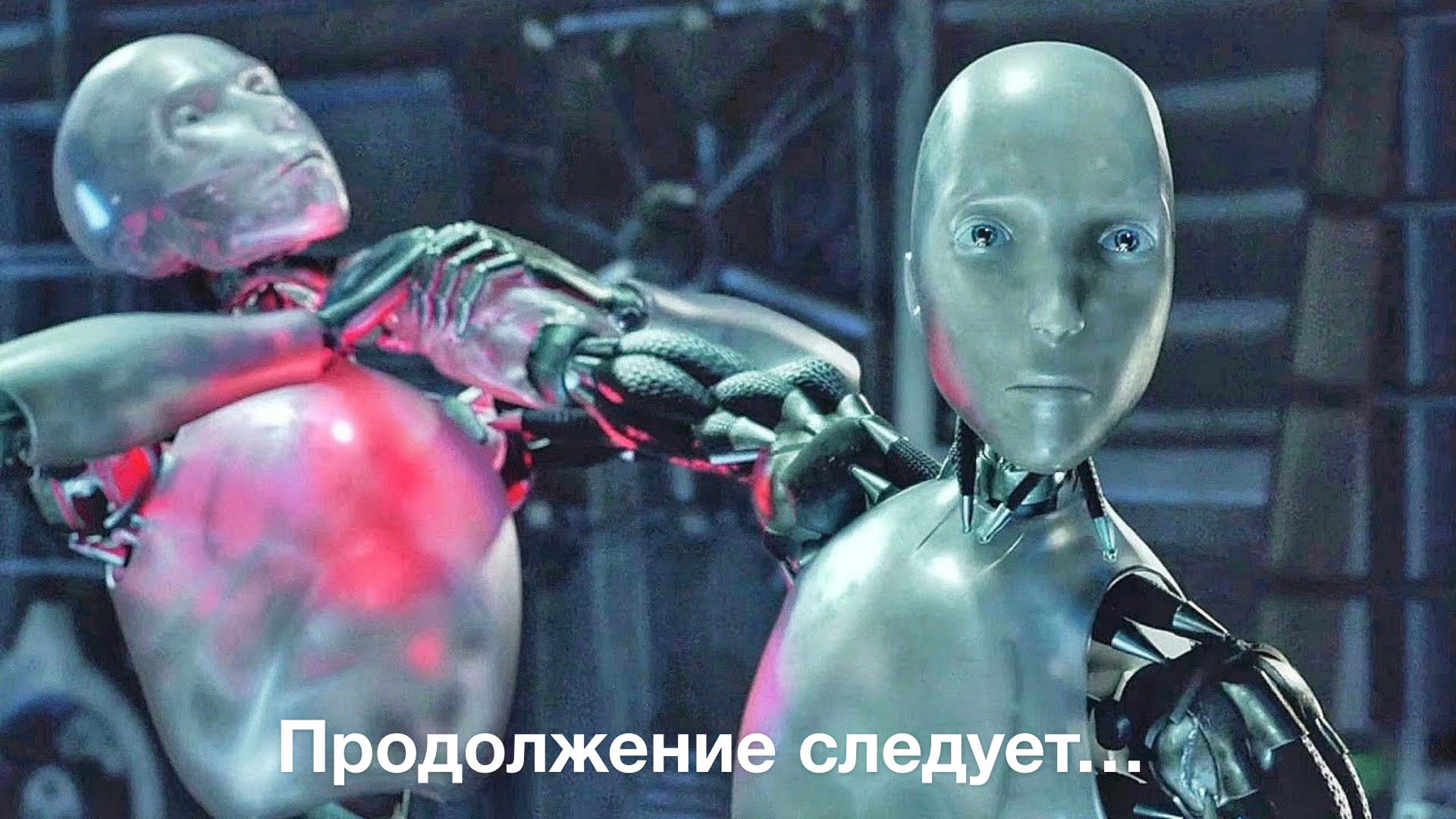
```
~SomeClass(){ ... };
```

- Имя то же что и имя класса, начинается с '~' - тильды.
- Не имеет параметров.
- Не возвращает значений.
- В конце всегда неявно вызывает деструкторы всех полей класса.

Вызов

- Неявно при выходе объекта класса из области видимости.
- При освобождении занимаемой памяти (вызове delete).
- Вручную. В этом случае удалит все поля класса, но не очистит память занимаемую самим классом.





Продолжение следует...